07

HAFTA

C# ile Nesneye Yönelik Programlama · Ders Notu

SOLID Prensipleri - II & Clean Code

(Temiz Kod için Temel Kurallar)

Bu Haftanın Kazanımları

Bu dersin sonunda öğrenciler:

- ✓ **ISP (Interface Segregation Principle — Arabirim Ayrımı İlkesi)** ile şişkin arabirimleri istemci odaklı küçük arabirimlere böler.
- ✓ **DIP (Dependency Inversion Principle — Bağımlılığın Tersine Çevrilmesi İlkesi)**'ni uygular; DIP, **Dependency Injection (bağımlılık enjeksiyonu)** ve **IoC (Inversion of Control — kontrolün tersine çevrilmesi)** arasındaki farkı açıklar.
- ✓ Constructor injection ile test edilebilir sınıflar tasarlar ve sahte nesnelerle test eder.
- ✓ **Clean Code (temiz kod)** yaklaşımıyla anlamlı isimler seçer ve C# isimlendirme kurallarını uygular.
- ✓ Uzun metotları tek iş yapan küçük fonksiyonlara böler, gereksiz yorumları kendini açıklayan koda dönüştürür.

İçindekiler

1. ISP (Interface Segregation)
2. DIP (Dependency Inversion)
3. Clean Code: Anlamlı İsimlendirme, Küçük Fonksiyonlar, Yorum Satırı Azlığı

— Haftanın Özeti

— Anahtar Kavramlar

— Alıştırmalar

— Ek Okuma Kaynakları

1. ISP (Interface Segregation)

> **TANIM** **Arabirim Ayrımı İlkesi**
>
> İstemciler, kullanmadıkları metotlara bağımlı olmaya zorlanmamalıdır. Tek bir büyük arabirim yerine, istemcilerin ihtiyaçlarına göre şekillenmiş birden fazla küçük arabirim tercih edilmelidir.

Çok sayıda ilgisiz metodu bir araya toplayan arabirimlere **fat interface (şişkin arabirim)** denir. Şişkin bir arabirimi uygulayan sınıf, desteklemediği metotları boş bırakmak ya da `NotImplementedException` fırlatmak zorunda kalır. Bu durum hem LSP'yi (6. hafta) ihlal eder hem de istemcileri hiç kullanmadıkları metotlardaki değişikliklere karşı savunmasız bırakır.

1.1 Kötü örnek: her şeyi yapan cihaz arabirimi

C# · CihazKotu.cs

```
public record Belge(string Ad);

public interface ICokFonksiyonluCihaz
{
    void Yazdir(Belge belge);
    void Tara(Belge belge);
    void FaksGonder(Belge belge, string numara);
    void Zimbala(Belge belge);
}

public class BasitYazici : ICokFonksiyonluCihaz
{
    public void Yazdir(Belge belge) => Console.WriteLine($"Yazdır: {belge.Ad}");

    // Bu cihaz aşağıdakilerin hiçbirini yapamıyor ama uygulamak zorunda:
    public void Tara(Belge belge) => throw new NotImplementedException();
    public void FaksGonder(Belge belge, string numara) =>
        throw new NotImplementedException();
    public void Zimbala(Belge belge) => throw new NotImplementedException();
}
```

1.2 Refactoring: rol odaklı küçük arabirimler

C# · CihazIyi.cs

```
public interface IYazici { void Yazdir(Belge belge); }
public interface ITarayici { void Tara(Belge belge); }
public interface IFaks { void FaksGonder(Belge belge, string numara); }

// Gerekirse küçük arabirimler birleştirilebilir
public interface ITarayiciYazici : IYazici, ITarayici { }

public class BasitYazici : IYazici
{
    public void Yazdir(Belge belge) => Console.WriteLine($"Yazdır: {belge.Ad}");
}

public class OfisMakinesi : ITarayiciYazici, IFaks
{
    public void Yazdir(Belge belge) { /* ... */ }
    public void Tara(Belge belge) { /* ... */ }
    public void FaksGonder(Belge belge, string numara) { /* ... */ }
}

// İstemci yalnızca ihtiyaç duyduğu yeteneği ister
public class FaturaCiktiServisi
{
    private readonly IYazici _yazici;
    public FaturaCiktiServisi(IYazici yazici) => _yazici = yazici;
    public void Bas(Belge fatura) => _yazici.Yazdir(fatura);
}
```

Arabirimleri, onları uygulayan sınıfın neler yapabildiğine göre değil, onları **kullanan istemcinin neye ihtiyaç duyduğuna** göre tasarlayın. Bu yaklaşımla ortaya çıkan arabirimlere **role interface (rol arabirimi)** denir.

1.3 Gerçek projeden bir örnek: okuma ve yazma ayrımı

Raporlama ekranı yalnızca veri okur; ancak 15 metotlu genel bir depo arabirimine bağımlı olursa `Sil` metodundaki bir değişiklik raporlama modülünü de etkiler. Okuma ve yazma sorumluluklarını ayırmak hem ISP'ye uyar hem de 12. haftada göreceğimiz mimari yaklaşımların temelini oluşturur:

C# · Depolar.cs

```
public interface IOkumaDeposu<T>
{
    T? IdIleGetir(int id);
    IReadOnlyList<T> TumunuGetir();
}

public interface IYazmaDeposu<T>
{
    void Ekle(T varlik);
    void Guncelle(T varlik);
    void Sil(int id);
}

// Raporlama servisi silme yetkisine sahip olmayan bir arabirim alır
public class NotRaporuServisi
{
    private readonly IOkumaDeposu<Ogrenci> _ogrenciler;
    public NotRaporuServisi(IOkumaDeposu<Ogrenci> ogrenciler) =>
        _ogrenciler = ogrenciler;
}
```

> **BİLGİ** **.NET koleksiyon arabirimleri ISP'nin ders kitabı örneğidir**
>
> `IEnumerable<T>` yalnızca dolaşmayı, `IReadOnlyCollection<T>` buna `Count`'u, `IReadOnlyList<T>` indeksleyiciyi, `ICollection<T>` ise ekleme/silmeyi getirir. Metot parametresi olarak her zaman **ihtiyacınızı karşılayan en dar arabirimi** isteyin: yalnızca dolaşacaksanız `List<T>` değil `IEnumerable<T>` alın.

2. DIP (Dependency Inversion)

> **TANIM** **Bağımlılığın Tersine Çevrilmesi İlkesi**
>
> 1. Üst seviye modüller alt seviye modüllere bağımlı olmamalıdır; her ikisi de soyutlamalara bağımlı olmalıdır.
> 1. Soyutlamalar ayrıntılara bağımlı olmamalıdır; ayrıntılar soyutlamalara bağımlı olmalıdır.

high-level module (üst seviye modül), uygulamanın iş kurallarını içeren kısımdır (ör. sipariş verme süreci). **low-level module (alt seviye modül)** ise veritabanı erişimi, e-posta gönderimi, dosya işlemleri gibi teknik ayrıntılardır. Geleneksel kodlamada iş kuralları doğrudan bu ayrıntılara bağımlı olur; oysa değerli ve kalıcı olan iş kurallarıdır, ayrıntılar ise sık değişir.

2.1 Kötü örnek: iş mantığı altyapıya kilitli

C# · SiparisServisiKotu.cs

```
public class SiparisServisi
{
    private readonly SqlSiparisDeposu _depo = new();          // somut sınıf!
    private readonly SmtpEpostaGonderici _eposta = new();     // somut sınıf!

    public void SiparisVer(Siparis siparis)
    {
        _depo.Ekle(siparis);
        _eposta.Gonder(siparis.MusteriEposta, "Siparişiniz alındı");
    }
}
```

Bu sınıf gerçek bir veritabanı ve e-posta sunucusu olmadan test edilemez. Veritabanını değiştirmek ya da e-posta yerine SMS göndermek istediğinizde, iş kuralını barındıran sınıfı açıp değiştirmeniz gerekir.

2.2 Refactoring: soyutlamalara bağımlılık

C# · SiparisServisi.cs

```
public record Siparis(int No, string MusteriEposta, List<string> Kalemler);

// Soyutlamalar, üst seviye modülün İHTİYAÇLARINA göre tanımlanır
public interface ISiparisDeposu { void Ekle(Siparis siparis); }
public interface IBildirimGonderici { void Gonder(string alici, string mesaj); }

// Üst seviye modül: yalnızca soyutlamaları tanır
public class SiparisServisi
{
    private readonly ISiparisDeposu _depo;
    private readonly IBildirimGonderici _bildirim;
```

C# · SiparisServisi.cs

```
    public SiparisServisi(ISiparisDeposu depo, IBildirimGonderici bildirim)
    {
        _depo = depo;
        _bildirim = bildirim;
    }

    public void SiparisVer(Siparis siparis)
    {
        if (siparis.Kalemler.Count == 0)
            throw new InvalidOperationException("Boş sipariş verilemez.");

        _depo.Ekle(siparis);
        _bildirim.Gonder(siparis.MusteriEposta, $"#{siparis.No} sipariş alındı.");
    }
}

// Alt seviye ayrıntılar: soyutlamaları uygular
public class SqlSiparisDeposu : ISiparisDeposu
{
    public void Ekle(Siparis siparis) { /* SQL INSERT */ }
}

public class SmtpEpostaGonderici : IBildirimGonderici
{
    public void Gonder(string alici, string mesaj) { /* SMTP */ }
}
```

Neden "tersine çevirme"? Arabirim, onu kullanan üst seviye modülün tarafında (iş katmanında) tanımlanır. Böylece bağımlılık okunun yönü değişir: artık iş mantığı altyapıya değil, altyapı iş mantığının tanımladığı sözleşmeye bağımlıdır.

Şema · Bağımlılık yönü

```
ÖNCE:   SiparisServisi ──────────► SqlSiparisDeposu
        (iş kuralı)                (altyapı ayrıntısı)

SONRA:  SiparisServisi ──► ISiparisDeposu ◄── SqlSiparisDeposu
        └──────── iş katmanı ────────┘          (altyapı katmanı)
```

2.3 DIP, DI ve IoC: üç farklı kavram

Bu üç kavram sıklıkla birbirine karıştırılır:

Kavram	Nedir?	Örnek
DIP	Bir tasarım **prensibi**: somut sınıflara değil soyutlamalara bağımlı ol	`SiparisServisi` → `ISiparisDeposu`
DI (Dependency Injection)	Bir **teknik**: bağımlılıkları sınıfın içinde oluşturmak yerine dışarıdan ver	Constructor injection
IoC (Inversion of Control)	Genel bir **kavram**: akışın kontrolünü framework'e bırakmak	ASP.NET Core'un controller nesnelerini kendisinin oluşturması
DI container	Bir **araç**: bağımlılıkları otomatik oluşturup enjekte eden kütüphane	`Microsoft.Extensions.DependencyInjection`

Bağımlılıklar üç yolla enjekte edilebilir: **constructor injection** (önerilen yöntem; bağımlılıklar zorunlu ve görünür olur), **property injection** (isteğe bağlı bağımlılıklar için) ve **method injection** (yalnızca tek bir metodun ihtiyaç duyduğu bağımlılıklar için).

2.4 Manuel DI ve test edilebilirlik

Nesnelerin birbirine bağlandığı tek noktaya **composition root (bileşim kökü)** denir. Konsol uygulamasında bu nokta `Program.cs`'tir; 11. haftada bu işi ASP.NET Core'un DI container'ına devredeceğiz.

C# · Program.cs

```
// Composition root: gerçek uygulamalar burada bağlanır
var servis = new SiparisServisi(new SqlSiparisDeposu(), new SmtpEpostaGonderici());
servis.SiparisVer(new Siparis(42, "ayse@ornek.com", ["Kitap", "Defter"]));
```

DIP'in en büyük kazancı test edilebilirliktir. Arabirimlerin sahte (**fake**) uygulamalarını yazarak iş kuralını veritabanı ve e-posta sunucusu olmadan doğrulayabiliriz:

C# · SiparisServisiTesti.cs

```
var depo = new SahteDepo();
var bildirim = new SahteBildirim();
var servis = new SiparisServisi(depo, bildirim);

servis.SiparisVer(new Siparis(1, "ali@ornek.com", ["Kitap"]));
Console.WriteLine(depo.Eklenenler.Count == 1);        // True
Console.WriteLine(bildirim.GonderimSayisi == 1);      // True

class SahteDepo : ISiparisDeposu
{
    public List<Siparis> Eklenenler { get; } = [];
    public void Ekle(Siparis siparis) => Eklenenler.Add(siparis);
}

class SahteBildirim : IBildirimGonderici
{
    public int GonderimSayisi { get; private set; }
    public void Gonder(string alici, string mesaj) => GonderimSayisi++;
}
```

> **BİLGİ** **Gerçek projelerde test**
>
> Profesyonel projelerde bu tür testler **xUnit**, **NUnit** veya **MSTest** gibi test framework'leriyle yazılır; sahte nesneler de **Moq** veya **NSubstitute** gibi kütüphanelerle tek satırda üretilir. DIP uygulanmamış bir kod tabanında bu araçların hiçbiri işe yaramaz.

> **İPUCU** **Primary constructor ile daha kısa DI kodu**
>
> C# 12'den itibaren sınıflar **primary constructor (birincil yapıcı)** alabilir: `public class SiparisServisi(ISiparisDeposu depo, IBildirimGonderici bildirim) { ... }`. Parametreler sınıfın tüm üyelerinde kullanılabilir. Dikkat: bu parametreler `readonly` değildir; değiştirilmemelerini garanti etmek isteyen ekipler onları yine `readonly` alanlara atar.

3. Clean Code: Anlamlı İsimlendirme, Küçük Fonksiyonlar, Yorum Satırı Azlığı

Robert C. Martin'in 2008'de yayımlanan *Clean Code* kitabı, yazılımcılık zanaatinin temel eserlerinden biridir. Kitabın çıkış noktası basittir: kod, yazıldığından çok daha fazla okunur. Bir satırı yazmaya harcanan zaman, onu anlamaya çalışan ekip arkadaşlarınızın (ve altı ay sonraki sizin) harcayacağı zamanın yanında önemsizdir. Temiz kod; okunabilir, sade, tek bir işi iyi yapan ve niyetini açıkça ortaya koyan koddur.

3.1 Anlamlı isimlendirme

İsimler, kodun en yaygın dokümantasyonudur. İyi bir isim; değişkenin neden var olduğunu, ne yaptığını ve nasıl kullanıldığını ek bir açıklamaya gerek bırakmadan anlatır.

İlke	Kötü	İyi
Niyeti açıklayan isimler	`int d; // geçen gün`	`int gecenGunSayisi;`
Kısaltma kullanmamak	`mstrNo`, `hsp`, `tmp`	`musteriNumarasi`, `hesap`, `geciciDosyaYolu`
Sihirli sayılar yerine sabitler	`if (yas >= 18)`	`if (yas >= ResitlikYasi)`
Tipi isme gömmemek (**Hungarian notation**)	`string strAd; int iSayac;`	`string ad; int sayac;`
Sınıflar isim, metotlar fiil	`class Hesapla`, `void Musteri()`	`class FiyatHesaplayici`, `void MusteriEkle()`
`bool` isimleri soru gibi okunmalı	`bool durum;`	`bool aktifMi;`, `bool odemeTamamlandi;`
Tutarlı kelime dağarcığı	`Getir`, `Al`, `Cek` karışık	Aynı anlam için hep aynı fiil: `Getir`
Yanıltıcı olmamak	`ogrenciListesi` (aslında `Dictionary`)	`ogrencilerNumarayaGore`

C# isimlendirme kuralları

Öğe	Kural	Örnek
Sınıf, struct, record, enum	PascalCase, isim	`OgrenciServisi`, `SiparisDurumu`
Interface	`I` önekli PascalCase	`IOgrenciDeposu`
Metot	PascalCase, fiil	`NotHesapla()`, `Kaydet()`
Property, event	PascalCase	`AdSoyad`, `StokDegisti`
Yerel değişken, parametre	camelCase	`toplamTutar`, `ogrenciId`
Private alan	`_` önekli camelCase	`_depo`, `_logger`
Sabit (`const`)	PascalCase	`MaksimumDenemeSayisi`
Asenkron metot	`Async` soneki	`GetirAsync()`
Generic tip parametresi	`T` önekli	`T`, `TKey`, `TEntity`

> **NOT** **Türkçe mi, İngilizce mi?**
>
> Tanımlayıcıların dili bir ekip kararıdır; önemli olan **tutarlılıktır**. Aynı dosyada `GetOgrenci()` gibi karışık isimlerden kaçının. Türkçe tercih edilse bile tanımlayıcılarda ı, ş, ğ, ü, ö, ç harflerini kullanmayın; C# bunlara izin verse de araçlarda, aramalarda ve farklı klavye düzenlerinde sorun çıkarır.

> **DİKKAT** **Türkçe "I" problemi**
>
> tr-TR kültüründe `"title".ToUpper()` sonucu `"TİTLE"`, `"FILE".ToLower()` sonucu ise `"fıle"` olur. Dosya uzantısı, protokol adı veya sözlük anahtarı gibi kültürden bağımsız metinleri karşılaştırırken `ToUpperInvariant()` ya da `string.Equals(a, b, StringComparison.OrdinalIgnoreCase)` kullanın.

3.2 Küçük fonksiyonlar

Clean Code'un fonksiyonlar için ilk kuralı küçük olmalarıdır; ikinci kuralı ise daha da küçük olmalarıdır. Bir fonksiyon:

- **Tek bir iş yapmalı** ve bu işi iyi yapmalıdır. Fonksiyonun içinden anlamlı isimli başka bir fonksiyon çıkarabiliyorsanız, o fonksiyon birden fazla iş yapıyordur.
- **Tek bir soyutlama seviyesinde** kalmalıdır. İş akışını anlatan bir metot ile döngü ve indeks hesabı yapan satırlar aynı yerde karışmamalıdır.
- **Az parametre** almalıdır: sıfır, bir veya iki ideal; üçten fazlası bir **parameter object (parametre nesnesi)** ihtiyacına işaret eder.
- `bool` bayrak parametresi almamalıdır: `Kaydet(ogrenci, true)` çağrısı fonksiyonun iki farklı iş yaptığını ilan eder.
- **side effect (yan etki)** içermemelidir: `SifreDogruMu()` adlı bir metot oturum da başlatmamalıdır.
- **Command-Query Separation (Komut-Sorgu Ayrımı)** ilkesine uymalıdır: bir metot ya bir şey yapar (komut) ya da bir soruya cevap verir (sorgu); ikisini birden yapmaz.

Önce: iç içe geçmiş, her şeyi yapan metot

C# · SiparisIsleKotu.cs

```
public string SiparisIsle(Siparis s)
{
    if (s != null)
    {
        if (s.Kalemler.Count > 0)
        {
            if (s.Musteri.Aktif)
            {
                decimal t = 0;
                foreach (var k in s.Kalemler) t += k.Fiyat * k.Adet;
                if (t > 1000) t = t * 0.9m;
                foreach (var k in s.Kalemler) k.Urun.Stok -= k.Adet;   // stok
                Console.WriteLine($"Mail: {s.Musteri.Eposta} {t}");    // e-posta
                return "OK";
            }
            else return "Müşteri pasif";
        }
        else return "Boş sipariş";
    }
    else return "Sipariş yok";
}
```

Sağa doğru kayan bu yapıya **arrow code (ok ucu kod)** denir. Koşulları takip etmek zordur, hata mesajları asıl kontrolden satırlarca uzaktadır ve metot en az dört farklı iş yapmaktadır.

Sonra: guard clause ve küçük fonksiyonlar

C# · SiparisIsleyici.cs

```
private const decimal IndirimEsigi = 1000m;
private const decimal IndirimOrani = 0.10m;

public SiparisSonucu SiparisIsle(Siparis? siparis)
{
    // Guard clause: geçersiz durumlar en başta elenir
    if (siparis is null) return SiparisSonucu.Hata("Sipariş yok");
    if (siparis.Kalemler.Count == 0) return SiparisSonucu.Hata("Boş sipariş");
    if (!siparis.Musteri.Aktif) return SiparisSonucu.Hata("Müşteri pasif");

    decimal toplam = IndirimliToplamHesapla(siparis);
    StoklariDus(siparis);
    OnayBildirimiGonder(siparis.Musteri, toplam);
    return SiparisSonucu.Basarili(toplam);
}

private static decimal IndirimliToplamHesapla(Siparis siparis)
{
    decimal toplam = siparis.Kalemler.Sum(k => k.Fiyat * k.Adet);
    return toplam > IndirimEsigi ? toplam * (1 - IndirimOrani) : toplam;
}

private static void StoklariDus(Siparis siparis)
{
    foreach (var kalem in siparis.Kalemler)
        kalem.Urun.Stok -= kalem.Adet;
}

private void OnayBildirimiGonder(Musteri musteri, decimal toplam) =>
    _bildirim.Gonder(musteri.Eposta, $"Siparişiniz alındı. Toplam: {toplam:C}");
```

C# · SiparisSonucu.cs

```
public record SiparisSonucu(bool BasariliMi, decimal Toplam, string? HataMesaji)
{
    public static SiparisSonucu Basarili(decimal toplam) => new(true, toplam, null);
    public static SiparisSonucu Hata(string mesaj) => new(false, 0m, mesaj);
}
```

Yeni `SiparisIsle` metodu bir hikâye gibi okunur: geçersiz durumları ele, toplamı hesapla, stokları düş, bildirimi gönder. Ayrıntı merak eden okuyucu ilgili küçük metoda iner; merak etmeyen, üst seviyedeki akışı anlamak için hiçbir satırı çözümlemek zorunda kalmaz.

> **İPUCU** **Refactoring'i IDE'ye yaptırın**
>
> Visual Studio'da bir değişken veya metodu yeniden adlandırmak için **Ctrl+R, Ctrl+R**, seçili satırlardan yeni metot çıkarmak için **Ctrl+R, Ctrl+M**, tüm hızlı düzeltme önerileri için **Ctrl+.** kısayollarını kullanın. IDE, adlandırmayı tüm referanslarda güvenle günceller.

3.3 Yorum satırı azlığı

Clean Code'un yorumlar hakkındaki tavrı nettir: kötü kodu yorumla açıklamaya çalışmayın, yeniden yazın. Kod değiştikçe yorumlar genellikle güncellenmez ve zamanla yalan söylemeye başlar. En iyi yorum, yazılmasına gerek kalmayan yorumdur.

C# · Burs.cs

```
// KÖTÜ: yorum, kodun ne yaptığını anlatmaya çalışıyor
// burs kontrolü: ortalama 3 üstü ve disiplin cezası yok
if (ogr.Ort > 3.0 && !ogr.Ceza && ogr.Donem >= 2)
    BursBaslat(ogr);

// İYİ: kod kendini anlatıyor
if (ogrenci.BursaUygunMu())
    BursBaslat(ogrenci);

// Ogrenci sınıfı içinde:
public bool BursaUygunMu() =>
    NotOrtalamasi > MinimumBursOrtalamasi && !DisiplinCezasiVar && Donem >= 2;
```

İlk örnekteki yoruma dikkat edin: kodda dönem şartı da var, ancak yorum bundan hiç söz etmiyor. Kod değişmiş, yorum güncellenmemiştir. Bu, yorumların neden güvenilmez olduğunun küçük ama tipik bir örneğidir.

Değerli yorumlar	Kaçınılması gereken yorumlar
Public API için XML dokümantasyonu (`///`)	Kodu tekrar eden yorumlar (`i++; // i'yi artır`)
Neden'i açıklayan yorumlar: iş kuralının kaynağı, alınan kararın gerekçesi	**Ne**'yi anlatan yorumlar (kod zaten anlatıyor)
Uyarılar: "Bu metot thread-safe değildir"	Yorum satırına alınmış eski kod (versiyon kontrolü zaten saklıyor)
Takip edilen geçici notlar: `// TODO: ...`	Günlük tutan yorumlar (`// 12.03 Ahmet değiştirdi`)
Karmaşık düzenli ifade (regex) açıklamaları	Kapanış parantezi yorumları (`} // if sonu`)

C# · NotHesaplayici.cs

```
/// <summary>
/// Öğrencinin ağırlıklı genel not ortalamasını (AGNO) hesaplar.
/// </summary>
/// <param name="dersler">Notu girilmiş dersler.</param>
/// <returns>0 ile 4 arasında bir değer; ders yoksa 0.</returns>
/// <remarks>
/// Harf notu katsayıları üniversite yönetmeliğindeki tabloya göre uygulanır.
/// </remarks>
public double AgnoHesapla(IReadOnlyList<DersNotu> dersler)
{
    if (dersler.Count == 0) return 0;
    double toplamKredi = dersler.Sum(d => d.Kredi);
    return dersler.Sum(d => d.Kredi * d.Katsayi) / toplamKredi;
}
```

3.4 Temiz kodun diğer ilkeleri

İlke	Anlamı
DRY (Don't Repeat Yourself — Kendini tekrarlama)	Her bilgi parçasının sistemde tek ve yetkili bir temsili olmalıdır. Kopyala-yapıştır kod, düzeltilmesi unutulan hatalar demektir.
KISS (Keep It Simple — Basit tut)	Çalışan en basit çözümü tercih edin; zekice ama anlaşılmaz koddan kaçının.
YAGNI (You Aren't Gonna Need It — Nasılsa ihtiyacın olmayacak)	Bugün gerekmeyen özelliği "belki lazım olur" diye yazmayın.

İlke	Anlamı
Boy Scout Rule (İzci kuralı)	Dokunduğunuz kodu bulduğunuzdan biraz daha temiz bırakın: bir isim düzeltin, bir metot ayırın.

Haftanın Özeti

- **ISP:** Şişkin arabirimler istemci ihtiyaçlarına göre küçük rol arabirimlerine bölünür. Metot parametrelerinde ihtiyacı karşılayan en dar arabirim istenir (`IEnumerable<T>`, `IReadOnlyList<T>`).
- **DIP:** İş kuralları altyapı ayrıntılarına değil, kendi tanımladığı soyutlamalara bağımlıdır. Arabirimin iş katmanında tanımlanması bağımlılık yönünü tersine çevirir.
- DIP bir prensip, DI bir teknik, IoC genel bir kavram, DI container ise bir araçtır. Constructor injection varsayılan tercihtir ve sahte nesnelerle test yapmayı mümkün kılar.
- **Anlamlı isimler** niyeti açıklar, kısaltma ve sihirli sayı içermez; C# isimlendirme kuralları (PascalCase, camelCase, `_alan`, `I` öneki, `Async` soneki) tutarlı biçimde uygulanır.
- **Küçük fonksiyonlar** tek iş yapar, tek soyutlama seviyesinde kalır, az parametre alır; guard clause'lar iç içe `if` yapılarını düzleştirir.
- **Yorumlar** "ne"yi değil "neden"i açıklamalıdır; public API'ler XML dokümantasyonu ile belgelenir.

Anahtar Kavramlar

Terim	Türkçe karşılığı	Kısa açıklama
fat interface	şişkin arabirim	İlgisiz pek çok metodu bir araya toplayan arabirim
role interface	rol arabirimi	İstemcinin ihtiyacına göre tasarlanmış küçük arabirim
high-level module	üst seviye modül	İş kurallarını içeren modül
low-level module	alt seviye modül	Veritabanı, e-posta gibi teknik ayrıntıları içeren modül
dependency injection	bağımlılık enjeksiyonu	Bağımlılıkların sınıfa dışarıdan verilmesi
inversion of control	kontrolün tersine çevrilmesi	Akışın kontrolünün framework'e bırakılması
composition root	bileşim kökü	Nesnelerin birbirine bağlandığı tek nokta
fake / mock	sahte nesne	Testlerde gerçek bağımlılığın yerine geçen nesne
naming convention	isimlendirme kuralı	Tanımlayıcıların yazımına dair ortak kurallar
guard clause	koruma ifadesi	Geçersiz durumları metodun başında eleyen erken dönüş
side effect	yan etki	Metodun beyan ettiği işin dışında durumu değiştirmesi
parameter object	parametre nesnesi	Çok sayıda parametreyi tek bir tipte toplama
arrow code	ok ucu kod	İç içe `if` bloklarıyla sağa kayan okunaksız kod

Alıştırmalar

1. `Giris`, `Cikis`, `SifreSifirla`, `RaporAl`, `KullaniciSil` ve `YetkiVer` metotlarını içeren `IKullaniciIslemleri` arabirimini; normal kullanıcı, yönetici ve raporlama modülünün ihtiyaçlarına göre ISP'ye uygun arabirimlere bölün.
2. İçinde `new HttpClient()` ve `File.WriteAllText` çağrıları bulunan bir `HavaDurumuServisi` yazın. Ardından `IHavaDurumuKaynagi` ve `IDepolama` soyutlamalarıyla DIP'e uygun hâle getirip sahte nesnelerle test edin.
3. DIP, DI ve IoC kavramlarını ders örnekleri dışındaki kendi senaryonuzla, birer paragrafla açıklayın.
4. Şu tanımlayıcıları anlamlı isimlerle değiştirin: `int d`, `List<int> lst`, `bool flag`, `void Proc()`, `string s2`, `var tmp`, `const int X = 86400`.
5. İndirim hesaplayan, stok kontrol eden, fatura oluşturan ve log yazan 50 satırlık tek bir metot yazın; ardından guard clause'lar ve en fazla 10 satırlık küçük metotlarla yeniden düzenleyin.
6. Kendi eski ödevlerinizden birindeki yorumları inceleyin: "ne"yi anlatan yorumları metot veya değişken ismine dönüştürün, public bir metot için XML dokümantasyonu yazın.
7. `CultureInfo.CurrentCulture = new CultureInfo("tr-TR")` ayarıyla `"index.html".ToUpper()` ve `"ID".ToLower()` sonuçlarını yazdırın; problemi invariant ve ordinal yöntemlerle çözün.

Ek Okuma Kaynakları

- C# kodlama kuralları
- C# tanımlayıcı isimlendirme kuralları
- .NET'te Dependency Injection
- XML dokümantasyon yorumları
- .NET'te string karşılaştırma için en iyi uygulamalar
- Martin Fowler — Inversion of Control Containers and the Dependency Injection pattern
- Kitap: Robert C. Martin — *Clean Code: A Handbook of Agile Software Craftsmanship*, Prentice Hall, 2008.